

Audit de sécurité - Application de réservation Ancrelieu (version Java / Spring Boot)

Contexte de l'audit

Ancrelieu nous a confié un audit de sécurité de l'application de réservation, dans le cadre du renouvellement de contrat avec l'un de ses clients. Cet audit porte sur la version Java (Spring Boot) de l'application, telle qu'elle est déployée actuellement en production.

L'audit a porté sur le système de comptes utilisateurs, la gestion des mots de passe, les mécanismes d'authentification et les contrôles d'accès aux données de réservation. Il ne couvre pas le déploiement ni la maintenance de l'application, qui font l'objet d'un chantier distinct.

Ce document liste les vulnérabilités constatées, par ordre de criticité décroissante, avec pour chacune une description en langage métier, le mécanisme technique en cause dans le code Spring Boot, et la protection attendue.

Synthèse des constats

Quatre vulnérabilités ont été identifiées. Les trois premières sont critiques : elles exposent directement les données personnelles et financières des clients d'Ancrelieu. La quatrième est à traiter mais présente un risque plus limité.

Vulnérabilité · Criticité · Composant concerné

Mots de passe stockés en clair · Critique · UserService / table users

Absence d'authentification sur les endpoints sensibles · Critique · ReservationController, UserController

Absence de contrôle d'autorisation entre utilisateurs (IDOR) · Critique · ReservationController

Absence de restriction sur les tentatives de connexion · Modérée · AuthController

Vulnérabilité 1 : mots de passe stockés en clair

En langage métier : si notre base de données était consultée par une personne non autorisée, que ce soit un employé indélicat ou un attaquant externe, elle lirait directement le mot de passe de chaque client. Beaucoup de clients réutilisent le même mot de passe sur plusieurs services, ce qui étend le risque bien au-delà d'Ancrelieu.

Constat technique : la classe UserService enregistre le mot de passe reçu du formulaire d'inscription directement dans le champ password de l'entité User, sans transformation. Aucun encodeur de mot de passe n'est déclaré dans la configuration Spring Security, et aucun bean PasswordEncoder n'est injecté dans le service.

Protection attendue : les mots de passe doivent être hachés avant écriture en base, avec un algorithme de hachage adapté et un coût de calcul paramétrable (l'écosystème Spring propose un encodeur BCrypt prêt à l'emploi via Spring Security Crypto). Le mot de passe en clair ne doit transiter qu'au moment de la connexion, jamais être conservé ni journalisé.

Vulnérabilité 2 : absence d'authentification sur les endpoints sensibles

En langage métier : n'importe qui, sans se connecter, peut aujourd'hui consulter la liste des réservations, créer une réservation au nom d'un tiers ou modifier une réservation existante en appelant directement l'adresse de l'API. Il suffit de connaître l'URL, ce qui est trivial à obtenir en inspectant le trafic du site.

Constat technique : la configuration de sécurité de l'application (SecurityFilterChain) autorise l'accès à l'ensemble des routes sans filtre d'authentification. Les endpoints `/api/reservations` et `/api/users` ne portent aucune annotation de sécurité, et aucun filtre ne vérifie la présence d'un jeton ou d'une session valide avant d'exécuter le contrôleur.

Protection attendue : chaque endpoint qui manipule des données de réservation ou de compte doit exiger une authentification préalable, mise en œuvre au niveau de la chaîne de filtres Spring Security (session ou jeton, selon ce que l'équipe retient). Les seuls endpoints publics doivent rester l'inscription et la connexion.

Vulnérabilité 3 : absence de contrôle d'autorisation entre utilisateurs (IDOR)

En langage métier : même une fois connecté avec son propre compte, un client peut aujourd'hui consulter ou modifier la réservation d'un autre client, simplement en changeant un identifiant dans l'adresse consultée. C'est le point que le client nous a explicitement signalé comme inacceptable avant renouvellement.

Constat technique : la méthode `getReservationById` du `ReservationController` retourne la réservation correspondant à l'identifiant demandé sans vérifier que cet identifiant appartient bien à l'utilisateur authentifié. Il s'agit d'une vulnérabilité de type IDOR (référence directe à un objet non protégée) : l'application fait confiance à l'identifiant fourni par le client sans le confronter au propriétaire réel de la ressource.

Protection attendue : chaque accès à une réservation doit être conditionné à la comparaison entre l'identifiant de l'utilisateur authentifié et le propriétaire de la ressource demandée, avant tout renvoi de données. Ce contrôle doit être testé explicitement, avec une tentative d'accès à la réservation d'un autre utilisateur qui doit être refusée.

Vulnérabilité 4 : absence de restriction sur les tentatives de connexion

En langage métier : rien n'empêche aujourd'hui un script automatisé d'essayer des milliers de mots de passe à la suite sur le même compte, jusqu'à en trouver un qui fonctionne.

Constat technique : l'endpoint `/api/auth/login` ne limite pas le nombre de tentatives par compte ou par adresse d'origine. Aucun verrouillage temporaire ni journalisation des échecs répétés n'est en place.

Protection attendue : ce point est à traiter si le temps le permet, mais il n'est pas bloquant pour le renouvellement du contrat. Il peut être écarté de ce chantier à condition que ce choix soit justifié et documenté.

Ce qui n'entre pas dans le périmètre de cet audit

Cet audit ne couvre ni le chiffrement des données au repos au niveau de la base de données, ni la

configuration réseau, ni les aspects de déploiement ou de journalisation applicative. Ces points seront traités dans un chantier ultérieur, distinct de la mise en conformité demandée par le client.